

Mul 算子测试报告

测试日期：2026-04-12

测试范围：acInnMul、acInnMuls、acInnInplaceMul、acInnInplaceMuls

覆盖指标：行覆盖率、函数覆盖率、分支覆盖率

一、测试策略

1.1 覆盖维度设计

维度	覆盖内容	实现状态
数据类型	FP32、FP64、FP16、BF16、INT32、INT8、UINT8、INT16、INT64、BOOL、COMPLEX64、COMPLEX128	完全覆盖
计算模式	tensor-tensor乘法、tensor-标量乘法	完全覆盖
API类型	普通接口、inplace接口、broadcast接口	完全覆盖
Shape场景	相同shape、广播、空维、高维、非2次幂	部分覆盖
数值范围	正常值、零、负数、边界值、极值	部分覆盖
特殊值	NaN、Inf、下溢边界	部分覆盖
异常输入	非法dtype、shape不匹配	部分覆盖

1.2 覆盖策略说明

- Dtype 多样性**：通过16种数据类型组合覆盖不同计算路径
- 广播机制**：标量广播、中间维广播、多维广播验证
- 混合类型**：FP16-FP32、INT32-FP32、COMPLEX数值混合测试
- Inplace变体**：独立实现路径覆盖
- 边界情况**：Inf×0、下溢、空维处理
- 特定条件分支**：部分RegBase/非RegBase路径、特定dtype推导尚不充分

二、测试用例概览

2.1 测试用例统计

总计：44 个测试用例

按类型分布

类型	用例数	用例编号
acInnMul (Tensor-Tensor)	18	1-18
Mul 广播	4	19-22
Mul 数值边界	2	23-24
acInnMuls (Tensor-Scalar)	13	25-37
acInnMuls 负向	1	38
acInnMuls FP16/BF16	2	39-40
acInnInplaceMul	1	41
acInnInplaceMul 广播	1	42
acInnInplaceMuls	2	43-44

Mul 测试用例详情 (1-18)

编号	用例名称	Dtype组合	特点	预期
int32_mul	基础INT32乘法	INT32×INT32→INT32	整数计算	PASS
double_mul	基础DOUBLE乘法	DOUBLE×DOUBLE→DOUBLE	高精度浮点	PASS
mixed_int32_float_mul	混合类型	INT32×FP32→FP32	不兼容组合	FAIL（预期）
int8_mul	低精度整数	INT8×INT8→INT8	溢出可能	PASS
mixed_float16_float32_mul	FP16混合	FP16×FP32→FP32	精度提升	PASS
1_BF16_Same	BF16同类型	BF16×BF16→BF16	低精度浮点	PASS
2_BF16_Mixed	BF16混合	BF16×FP32→FP32	精度提升	PASS
3_BF16_Mixed_Rev	BF16反向混合	FP32×BF16→FP32	精度提升	PASS
4_FP16_Same	FP16同类型	FP16×FP16→FP16	中等精度浮点	PASS
5_FP16_Mixed	FP16混合	FP16×FP32→FP32	精度提升	PASS
6_FP16_Mixed_Rev	FP16反向混合	FP32×FP16→FP32	精度提升	PASS
7_FP32_Same	FP32同类型	FP32×FP32→FP32	标准浮点	PASS
8_INT32_Same	INT32同类型	INT32×INT32→INT32	整数计算	PASS
9_UINT8_Same	UINT8同类型	UINT8×UINT8→UINT8	无符号整数	PASS
10_INT8_Same	INT8同类型	INT8×INT8→INT8	有符号低精度	PASS
11_INT64_Same	INT64同类型	INT64×INT64→INT64	大整数	PASS
12_INT16_Same	INT16同类型	INT16×INT16→INT16	中等整数	PASS
18_Illegal_UINT32	非法dtype（负向）	UINT32×UINT32→UINT32	不支持类型	FAIL(预期)

Mul 高级场景（19-24）

编号	用例名称	场景描述	验证目标
19_FP32_NonPowerOfTwo_SameShape	非2次幂Shape	{33,65}	不同Tiling分支
20_ScalarLike_Broadcast	标量广播	{2,3}×{1}→{2,3}	广播机制
21_Broadcast_MiddleDim	中间维广播	{2,1,4}×{2,3,4}→{2,3,4}	多维广播
22_EmptyDim_Broadcast	空维处理	{2,0,3}×{1,3}→{2,0,3}	边界处理
23_FP32_Underflow	下溢检测	FLT_MIN×1e-20	精度极限
24_Inf_Times_Zero	Inf×0→NaN	INF×0	IEEE规则

Complex数值测试（Mul部分）

编号	用例名称	Dtype	特点
13_Complex32_Same	COMPLEX32×COMPLEX32	COMPLEX32	低精度复数
14_Complex64_Same	COMPLEX64×COMPLEX64	COMPLEX64	标准复数
14_FP32_Complex64_Mixed	FP32×COMPLEX64	混合	复数促进
14_Complex64_FP32_Mixed	COMPLEX64×FP32	混合	复数促进
14_FP32_Complex64_Mixed_Rev2	FP32×FP32→COMPLEX64	特殊	输出促进
15_Complex128_Same	COMPLEX128×COMPLEX128	COMPLEX128	高精度复数
15_FP64_Complex128_Mixed	FP64×COMPLEX128	混合	复数促进
16_BOOL_Same	BOOL×BOOL	BOOL	逻辑运算
17_Double_Same	DOUBLE×DOUBLE	DOUBLE	64位浮点

Muls 测试用例（25-40）

编号	用例名称	Dtype组合	特点
25_Double_Muls	DOUBLE×标量	DOUBLE	高精度
26_Float_Muls_ZeroScalar	FP32×0标量	FP32	零值
27_Float_COMPLEX64_Muls	FP32×COMPLEX64标量	混合	复数标量
28_Complex64_Float_Muls	COMPLEX64×FP32标量	混合	复数张量
29_Complex64_Muls	COMPLEX64×COMPLEX64标量	COMPLEX64	纯复数
30_Float_Complex_Muls_Mixed	FP32×FP32标量→COMPLEX64	特殊	输出促进
31_Int64_Complex_Muls_Mixed	INT64×COMPLEX64标量	混合	整数×复数
32_Complex_Int_Muls_Mixed	COMPLEX64×INT64标量	混合	复数×整数
33_Complex128_Float_Muls	COMPLEX128×FP32标量	混合	高精度复数
34_Complex128_Double_Muls	COMPLEX128×DOUBLE标量	混合	高精度混合
35_Complex128_BOOL_Muls	COMPLEX128×BOOL标量	混合	复数×逻辑
36_Complex128_Muls	COMPLEX128×COMPLEX128	COMPLEX128	纯复数
37_Int16_Muls	INT16×INT16标量	INT16	整数标量
38_Muls_Illegal_UIINT32	UIINT32（负向）	非法	异常处理
39_Muls_FP32_FP16Scalar	FP32×FP16标量	FP混合	低精度标量
40_Muls_FP32_BF16Scalar	FP32×BF16标量	FP混合	低精度标量

Inplace 测试用例（41-44）

编号	用例名称	API	特点
41_Int32_InplaceMul	acInnInplaceMul	INT32×INT32	基础inplace
42_Broadcast_InplaceMul	acInnInplaceMul	{2,3}×{3}	广播inplace
43_Double_InplaceMuls	acInnInplaceMuls	DOUBLE×标量	inplace标量
44_Int32_InplaceMuls	acInnInplaceMuls	INT32×标量	inplace标量

三、覆盖率分析

3.1 代码覆盖率统计

文件	行覆盖率	函数覆盖率	分支覆盖率
aclnn_mul.cpp	76.5%	94.1%	27.2%
mul.cpp	84.6%	100%	43.7%
mul_tiling_arch35.cpp	89.2%	95.1%	28.4%

3.2 未覆盖的代码总结

模块	未覆盖内容
CheckMulsDtype	Dtype检查错误分支
InferTensorScalarDtype	非RegBase路径
GetCastedFloat/IsFloatEqual	FP16/BF16标量处理
CheckMulsPromoteDtype	类型推导失败分支
mul_infershape.cpp	整个文件

附录A：测试执行命令

第一步：编译算子（启用覆盖率插桩）

```
cd /home/workspace/ops-math
bash build.sh --pkg --soc=ascend950 --ops=mul --vendor_name=custom --cov
```

编译成功后会在 `build_out/` 目录下生成算子安装包。

第二步：安装算子包

```
./build_out/cann-ops-math-custom_linux-x86_64.run
```

第三步：运行测试（使用 CPU 模拟器）

```
bash build.sh --run_example mul eager cust \\\
--vendor_name=custom --simulator --soc=ascend950 --cov
```

运行成功后会输出算子的计算结果，同时在 `build/` 目录下生成覆盖率数据文件（`.gcda`）。

第四步：查看覆盖率

```
# 查找 mul 算子相关的覆盖率数据文件
find build -name "*.gcda" | grep mul

# 对指定文件生成覆盖率报告，例如：
gcov -b -c
build/math/mul/CMakeFiles/ophost_math_tiling_obj.dir/op_host/arch35/mul_tiling_arch35.cpp.gcda
```

附录B：覆盖率结果截图

